

DFT IDE

DFT IDE 插件业务规划

从 demo 验证走向项目级 DFT 工作台

把分散在 Git、OBS、脚本、终端和本地目录里的
DFT 工作，收束到 VS Code 中的统一业务入口。

本材料基于当前 demo 界面与能力，规划后续产品化方向、用户场景、价值闭环和落地节奏。

1. 业务判断：为什么值得继续做

DFT 项目常见痛点不是单点工具缺失，而是流程上下文分散：项目目录靠口口相传，公共路径与工具版本散落在文档中，设计与验证配置重复填写，执行结果和日志诊断难以沉淀，跨仓库协作缺少统一入口。

DFT IDE 的业务定位，是把 VS Code 从代码编辑器扩展成“本地 DFT 工作台”：在不替换现有 GitLab、OBS、集群和脚本体系的前提下，为工程师提供项目准入、配置编排、执行协同和结果追踪的一致体验。

降本	提效	控风险	可沉淀
减少环境准备和路径排查时间	把设计/验证流程变成可复用步骤	用准入、状态和日志诊断提前暴露问题	把配置、执行、结果变成项目资产



示意图基于上传截图重绘：项目首页、项目列表、本地状态目录与准入模型构成第一层业务入口。

2. 用户与核心使用场景

插件不是给单一角色使用的配置页，而是围绕项目生命周期服务多个角色。它的价值在于让每个角色只看见自己当下要完成的动作，同时共享同一份项目上下文。

项目负责人	从项目平台进入项目，确认仓库、本地根目录、成员和阶段状态。	关注项目准入、进展可见、配置可追溯。
DFT 设计工程师	维护设计树，配置 hibist/sailor/DCG 等设计流程，提交设计仓库。	关注少填错、能复用、能提交。
验证工程师	复用设计树与公共配置，生成 PLAN 脚本，执行仿真并查看问题定位。	关注脚本生成、终端执行、日志诊断。
平台/IT 团队	维护 OBS、Git、集群队列、权限与工具版本映射。	关注标准化、权限、审计和稳定性。
评审/管理者	查看项目流转、关键配置状态、执行结果和质量信号。	关注效率指标和风险可视化。

优先建设的 5 个场景

01 项目初始化	从项目列表进入，自动建立本地目录、映射 data/design/verification 仓库，并初始化本地状态。
02 公共配置	一次维护设计树、归一化表格、OBS 公共数据和同步目标，设计/验证共用。
03 模块级设计流	按设计树选择模块，逐步完成路径模板、工具版本、执行页、结果页和端云同步。
04 验证执行与诊断	生成脚本、打开 VS Code 终端、解析日志，并把错误/警告落到 Problems 面板。
05 协同归档	把配置变更 commit/push 到对应仓库，形成可审计的项目变更记录。

3. 产品主线：从配置页面到 workflow 闭环

demo 已经验证了一个关键方向：用户不需要先理解插件所有能力，只需要沿着项目、公共配置、Design、Verification 的导航完成任务。后续规划应继续强化“流程化”和“上下文继承”。



产品层	规划重点	用户感知
项目入口	项目清单、角色、阶段、GitLab 三仓映射、本地目录托管。	一键进入当前项目，不再手工找路径。
公共配置	设计树、公共路径、OBS 数据、归一化表格和跨流程同步。	公共信息只配置一次。
workflow 页面	Design/Verification 按步骤推进，并保留每个模块的局部状态。	像填流程单一样完成 DFT 任务。
执行与结果	终端执行、日志解析、问题定位、历史记录、报告入口。	执行失败能快速知道在哪里失败。
协同归档	Git commit/push、远端同步、OBS 查看器、后续云端看板。	交付物可追踪、可复盘。

4. 能力规划：围绕项目资产，而不是围绕按钮

后续功能优先级建议从“能否沉淀项目资产、减少跨工具切换、降低交付风险”来判断，而不是单纯补齐页面控件。

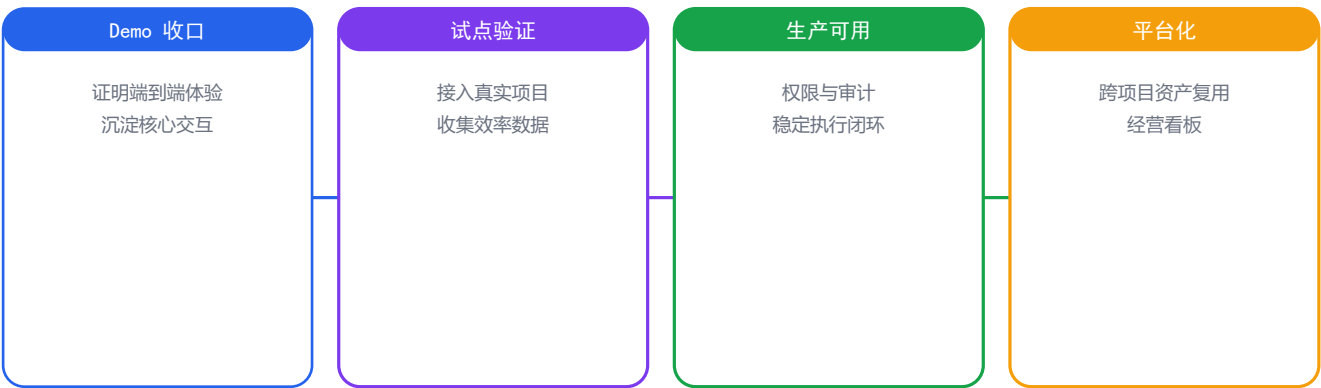
能力域	近期建设内容	业务收益
项目准入	项目平台对接、项目角色、仓库映射、本地根目录初始化、最近项目恢复。	降低新项目进入成本，减少环境差异。
配置资产	公共配置模板、设计树导入/编辑、模块级配置继承、配置差异提示。	形成可复用模板，减少重复录入。
流程编排	Design/Verification 统一 FlowShell、步骤状态、校验规则、执行历史。	把经验固化为流程，降低新人上手成本。
执行协同	终端命令生成、队列/资源参数、任务提交/取消、日志回收。	让执行过程可控，可中断，可复盘。
质量反馈	日志解析、Problems 面板、报告定位、阻断规则、准出检查。	问题更早暴露，评审有依据。
端云一体	Git、OBS、项目看板、执行上传、权限和审计。	把本地效率和组织治理连接起来。

截图给出的业务信号

首页说明项目准入和本地托管已经成型；Common 页说明跨流程公共资产已经有入口；Design 页说明模块级配置和步骤化导航可行；Verification 页说明执行、终端、Problems 诊断可以进入闭环；Git 同步页说明协作交付已经具备雏形。

5. 路线图：从试点可用到平台化经营

建议采用“小范围真实项目试点 - 收敛核心流程 - 再扩展平台能力”的节奏。先把设计与验证两个高频闭环跑顺，再补 Formal/STA 和组织级看板。



阶段	核心目标	交付物	验收指标
Demo 收口	证明工作台体验完整。	项目入口、Common、Design、Verification、Git/OBS/终端演示链路。	核心场景可演示，关键数据可保存。
试点验证	接入 1-2 个真实项目。	真实仓库映射、真实设计树、真实脚本与日志样例。	环境准备时间下降，配置返工减少。
生产可用	可靠地支撑日常项目。	权限、审计、错误恢复、执行历史、配置模板库。	周活用户、成功执行率、问题定位时间达标。
平台化	支撑多项目经营和资产复用。	项目看板、指标分析、模板市场、Formal/STA 扩展。	跨项目复用率和质量趋势可视化。

6. 运营与落地：让插件真的被用起来

业务成功不只取决于功能完成度，还取决于上线方式。DFT IDE 适合以“种子项目 + 标准模板 + 培训素材 + 指标复盘”的方式推广。

落地抓手	建议做法
种子用户	选择一个设计团队和一个验证团队试点，覆盖项目负责人、资深工程师和新人。
模板资产	把常用路径、工具版本、队列资源、报告位置和提交规范整理成项目模板。
培训材料	按“进入项目 - 公共配置 - Design 配置 - Verification 执行 - 提交归档”录制 10 分钟以内演示。
指标复盘	跟踪配置耗时、执行成功率、日志定位耗时、模板复用率、Git 提交完整率。
治理边界	先做流程入口和状态沉淀，不急于替代后端调度、企业权限或现有数据平台。

建议下一步

- 1. 固化 demo 中已跑通的端到端故事线，形成一套标准演示项目。
- 2. 与真实项目 owner 确认仓库映射、本地目录、OBS 和集群字段的最小必填项。
- 3. 选定 Design 和 Verification 各 1 条主流程做试点，收集效率与返工数据。
- 4. 以配置资产和执行历史为核心，准备生产可用版本的权限、审计和错误恢复设计。

一句话定位

DFT IDE 是面向芯片 DFT 项目的本地工作台，用统一入口承接项目准入、配置编排、执行诊断和协同归档，让工程经验变成可复用的项目资产。